

## Introduction

---

Every Python interview eventually circles back to one deceptively simple question: "Why didn't you use a set here instead of a list?" It sounds like a style nitpick, but it isn't. Picking the wrong container doesn't just make your code look less polished — it can silently produce wrong results or quietly tank your program's performance as the input grows.

This is the kind of thing interviewers test because it separates someone who can make code *work* from someone who understands *why* it works. Lists, tuples, and sets all hold groups of values, but they encode three completely different promises about that data: can it change, does order matter, and are duplicates allowed. Once you internalize those three questions, choosing the right structure becomes automatic instead of something you second-guess mid-interview.

We'll rebuild tuples and sets from the ground up — creation, unpacking, the operations that make each one shine — and dig into a bug that trips up almost every beginner at least once: trying to put a list inside a set.

## Problem Overview

---

Python gives you three built-in ways to store a collection of items: `list`, `tuple`, and `set`. On the surface they look interchangeable — all three can hold multiple values, all three can be looped over, all three support things like `len()`. But underneath, each one makes a different trade-off:

- **List** — ordered, changeable (mutable), duplicates allowed.
- **Tuple** — ordered, unchangeable (immutable), duplicates allowed.
- **Set** — unordered, changeable, duplicates automatically removed.

The task isn't to memorize these three bullet points — it's to understand *why* those trade-offs exist and how to recognize, from the shape of your data, which one you actually need.

## Example

---

Suppose you're modeling a single point on a map:

```
point = (40.7128, -74.0060) # tuple: a fixed coordinate pair
```

This never needs to change — a coordinate is a coordinate. A tuple signals that intent directly.

Now suppose you're tracking which user IDs have already voted in a poll:

```
voted = set()
voted.add(101)
voted.add(203)
voted.add(101) # ignored, 101 is already there
print(voted)   # {203, 101} – order not guaranteed, no duplicate
```

The second `add(101)` does nothing. No error, no warning — it just vanishes, because a set's entire job is to guarantee uniqueness.

And a list, for comparison, is what you reach for when the collection needs to grow, shrink, and hold duplicates in a specific order:

```
scores = [88, 92, 88, 75] # order and duplicates both matter here
```

## Intuition

---

Here's how an engineer actually arrives at the right structure, instead of defaulting to a list out of habit.

Start by asking three questions about the data:

1. **Will this data ever need to change after creation?** If no, that's a strong signal for a tuple. Locking it down isn't a limitation — it's documentation. Anyone reading `point = (x, y)` instantly knows this is a fixed record, not something to mutate mid-program.
2. **Do I care about order, or do duplicates even make sense?** If you're just tracking "have I seen this before," order is irrelevant and duplicates are actively unwanted. That's exactly what a set is built for.
3. **Am I going to check membership over and over?** This is the question people miss most often. A list checks membership by walking through every item one at a time —  $O(n)$ . A set uses hashing to jump almost directly to the answer —  $O(1)$  on average. If you're doing `if x in collection` inside a loop, a set will beat a list every single time as the data grows.

The mental model: a list is your everyday flexible container. A tuple is that same ordered idea, frozen solid. A set throws away order entirely in exchange for uniqueness and near-instant lookups.

## Brute Force Solution

---

There isn't really a "brute force vs optimal" split here in the algorithmic sense — this is a data structure selection problem, not a search problem. But it's useful to frame it that way, because the "brute force" instinct is: **use a list for everything**.

**Idea:** Store every collection as a list, regardless of whether it needs to change, whether order matters, or whether duplicates should be allowed.

**Algorithm:** 1. Create a list. 2. Append values as needed. 3. Check membership with `in`. 4. Deduplicate manually with loops or extra tracking if needed.

**Advantages:** - Simple, one structure to remember. - Works for every use case, technically.

**Disadvantages:** - No protection against accidental mutation of "fixed" data. - Manual deduplication is extra code you have to write and maintain. - Membership checks are slow —  $O(n)$  per check. - Code doesn't communicate intent. A reader can't tell if a list is meant to change or not.

**Time complexity:**  $O(n)$  per membership check,  $O(n)$  for manual dedup. **Space complexity:**  $O(n)$ .

## Optimal Solution

The optimal approach is to match the structure to the data's actual constraints:

| Question                                     | Answer      | Structure                          |
|----------------------------------------------|-------------|------------------------------------|
| Does it need to change?                      | No          | Tuple                              |
| Does it need to change?                      | Yes         | List or Set                        |
| Do duplicates matter, and does order matter? | Yes to both | List                               |
| Do duplicates matter, and does order matter? | No to both  | Set                                |
| Will I check membership repeatedly?          | Yes         | Set (hashing gives $O(1)$ lookups) |

## Tuple creation and unpacking

```
person = ("Maya", 29)
name, age = person # unpacking – order lines up exactly
```

The one-item tuple trap: `(5)` is just the integer `5` in parentheses. You need a trailing comma — `(5,)` — to actually make it a tuple. Python has no other way to tell the difference.

## Set creation and operations

```
empty = set() # NOT {} – that creates an empty dict
a = {1, 2, 3}
b = {2, 3, 4}
```

```
a | b # union: {1, 2, 3, 4}
a & b # intersection: {2, 3}
a - b # difference: {1}
```

## The hashing rule

Sets (and dictionary keys) require every element to be **hashable** — meaning Python can compute a fixed "barcode" for it that never changes. Numbers, strings, and tuples are hashable because they're immutable. Lists are not, because their contents can change after the hash would have been computed, which would break the whole lookup mechanism.

```
bad_set = {[1, 2]} # TypeError: unhashable type: 'list'
good_set = {(1, 2)} # fine – tuples are hashable
```

## Python Code

---

```
def unique_pairs(records: list[tuple[str, int]]) -> set[tuple[str, int]]:
    """Return unique (name, age) records using a set for dedup."""
    return set(records)

def shared_users(group_a: list[int], group_b: list[int]) -> set[int]:
    """Return user IDs present in both groups using set intersection."""
    return set(group_a) & set(group_b)

def fast_lookup_demo(candidates: list[int], target: int) -> bool:
    """Membership check backed by a set for O(1) average lookup."""
    seen = set(candidates)
    return target in seen
```

## Code Walkthrough

---

- `unique_pairs` converts a list of tuples into a set. Tuples are hashable, so this works and automatically drops duplicate records — no manual loop needed.
- `shared_users` converts both lists to sets and uses `&` for intersection, replacing what would otherwise be a nested loop comparing every element of one list against every element of the other.
- `fast_lookup_demo` builds a set once, then does the membership check with `in`. That single line is O(1) on average instead of O(n), because the set hashes `target` and jumps straight to its bucket instead of scanning.

## Dry Run

---

Take `shared_users([1, 2, 3, 4], [3, 4, 5, 6])`.

1. `set(group_a) → {1, 2, 3, 4}`
2. `set(group_b) → {3, 4, 5, 6}`
3. `&` compares hash buckets rather than scanning element by element: both sets contain `3` and `4`.
4. Result: `{3, 4}`.

Compare that to the list version, which would need a nested loop checking every element of `group_a` against every element of `group_b` — 16 comparisons instead of near-instant hash lookups.

## Complexity Analysis

---

- **Set membership check:**  $O(1)$  average time, because hashing maps a value directly to a bucket instead of scanning. Worst case  $O(n)$  if many values collide into the same hash bucket, but Python's hash design makes this rare in practice.
- **Set intersection/union/difference:**  $O(\min(\text{len}(a), \text{len}(b)))$  roughly, since it iterates the smaller set and checks membership in the other.
- **List membership check:**  $O(n)$ , since every element must be checked in the worst case.
- **Space:**  $O(n)$  for all three structures — a set needs extra space internally for its hash table, but this is still linear in the number of elements.

These are correct because they follow directly from the underlying implementation: sets are backed by hash tables, lists are backed by contiguous arrays.

## Alternative Solutions

---

- **frozenset** — an immutable version of a set. Useful when you need set operations (union, intersection) but also need the result to be hashable itself, such as using a group of items as a dictionary key.
- **Dictionary instead of a set** — if you catch yourself building a set just to check "does this key exist," but you also need to store a *value* associated with each key, that's a sign you actually need a `dict`, not a `set`. This is one of the most common misuses called out in the video.

## Edge Cases

---

- **Empty collection:** `set()` for an empty set — `{}` gives you an empty dict instead, a classic trap.
- **Single-item tuple:** `(5,)` requires the trailing comma; `(5)` is just an integer.
- **Duplicate values in a set:** silently ignored, no error.
- **Unhashable elements:** attempting `{[1, 2]}` raises `TypeError` immediately.
- **Very large collections:** this is exactly where the  $O(1)$  vs  $O(n)$  membership check gap becomes visible in real runtime.

## Common Mistakes

---

1. Using `{}` expecting an empty set — it's actually an empty dict.
2. Forgetting the trailing comma for a single-element tuple.
3. Trying to store a list inside a set (or as a dict key) and getting a `TypeError`.
4. Assuming a set preserves insertion order — it doesn't guarantee any order.
5. Using a list for repeated membership checks in a hot loop instead of converting to a set first.
6. Reaching for a set when you actually need key-value pairing, which calls for a dictionary instead.

## Interview Questions

---

- Why are tuples hashable but lists are not?
- When would you choose a `frozenset` over a regular set?
- What's the time complexity of checking membership in a list vs. a set, and why?
- How would you find duplicates in a list using a set?
- Why can a tuple be used as a dictionary key but a list cannot?

## Similar Problems

---

- **LeetCode 217 — Contains Duplicate:** directly tests set-based deduplication and membership checks.
- **LeetCode 349 — Intersection of Two Arrays:** a direct application of set intersection.
- **LeetCode 1 — Two Sum:** commonly solved using a dictionary (a natural extension once you understand hashing and sets).
- **LeetCode 242 — Valid Anagram:** tests understanding of counting and set/dict tradeoffs.

## Key Takeaways

---

Lists, tuples, and sets aren't interchangeable conveniences — each one encodes a promise about your data. A tuple says "this is fixed." A set says "I only care about uniqueness and fast lookups, not order." A list says "I need full flexibility." Choosing correctly makes your code both correct and fast, and it makes the intent obvious to the next person reading it — including future you.

## Watch the Video

---

For the full walkthrough with live coding, including the list-in-a-set crash happening in real time, watch the video here: {LINK}

## About the Series

---

This lesson is part of the **Daily Python LeetCode** series, where every video breaks down one core Python concept or interview problem — building the intuition first, then the code — so you understand *why* a solution works, not just how to type it out.

## Call To Action

---

If this cleared up something that's been silently confusing you about Python's data structures, tap like on the video — it's the easiest way to tell me it landed. Subscribe on YouTube so you don't miss the next lesson, and subscribe to the Substack for the full written breakdown delivered straight to your inbox. Drop a comment if you've been bitten by the list-in-a-set trap before, and share this with a fellow developer who defaults to lists for everything.

---

## Quick Review

---

### Why have tuple, set, AND list instead of just list?

Each optimizes a different need: tuple=fixed/immutable order, set=fast uniqueness/lookup, list=flexible ordered mutation.

### Set membership check ``x in my_set`` — time complexity?

$O(1)$  average, via hashing. Compare to ``x in my_list`` which is  $O(n)$ .

**Why can't you put a list inside a set?**

Lists are mutable/unhashable — sets require hashable elements. Use a tuple instead.

**Tuple unpacking: `a, b = (1, 2)` — what's this called and why use it?**

Unpacking; assigns each element to a variable in one line, common for returning multiple values from a function.

**Tuple vs List: when pick tuple?**

Data won't change (coordinates, RGB values) — signals intent, is immutable, and slightly faster/smaller.

**Interview follow-up: how do you dedupe a list while keeping order?**

``list(dict.fromkeys(my_list))`` — dict preserves insertion order and keys are unique (Python 3.7+).

---

Python Lesson 9: Tuples and Sets (Immutable sequences, set operations, and choosing the right collection) — free Python cheat sheet